



CineMax

Manuale tecnico

Laboratorio interdisciplinare A, AA25-26
Laurea triennale, Università degli Studi dell'Insubria

Autori del progetto:

- Leonardo Bruscoli, 760287
- Edo Hodzic, 761022
- Piergiorgio Tomaciello, 761013
- Tamirat Tucci, 756969

Sommario

Installazione e Setup	4
Installazione	4
Setup	4
Struttura e Funzionalità	5
1. Architettura a Layer	5
2. Layer Model	5
2.1 Interfacce di Contratto	5
Identifiable<E>	5
ItemInitializer<E>	5
2.2 Entità Persistibili	5
Movie	6
Show	6
Reservation	6
User	6
2.3 Entità di Presentazione	6
3. Layer Repository	7
3.1 Interfaccia FileRepository<F, ID>	7
3.2 GenericRepository<E, F>	8
3.2.1 Struttura del File e Parametri	8
3.2.2 Analisi della Complessità	8
3.2.3 Meccanismo di Lettura a Blocchi	9
3.2.4 Meccanismo di Modifica e Cancellazione	9
3.2.5 Cache del MaxId	9
3.3 Repository Specifici	10
3.4 Gestione delle Eccezioni	10
4. Layer Utility	10
4.1 OutputPrinter	10
4.2 PasswordHandler	11
4.3 SafetyException	11
5. Layer Service	11

5.1 Authentication	12
5.2 MovieService.....	12
5.3 ShowService	13
5.4 ReservationService.....	13
5.5 Scelta Architetture: Stampa in Streaming nel Layer Service	14
6. Layer TextUserInterface	15
6.1 MenuHelper	15
6.2 TextUserInterface	15
Ciclo Principale e Dispatch per Ruolo	16
Gestione delle Eccezioni	16
getAdminPanel()	16
7. Layer Cinemax (Main)	16
7.1 Sequenza di Inizializzazione	16
7.2 Costanti Globali	17
Limiti della Soluzione Sviluppata	18
Parsing Fragile	18
Lentezza degli Scorrimenti	18

Installazione e Setup

Installazione

L'applicazione è portable e può essere scaricata come zip dalla repository remota <https://github.com/Piguit/CineMax.git>. Dopo aver decompresso la cartella, è possibile navigare dentro CineMax-main e poi in bin, dove CineMax.jar è situato.

Cartella	Contenuto
bin	Packages compilati, CineMax.jar e relativo manifest
src	Packages con codice sorgente
data	File per la persistenza delle basi di dati
doc	Manuale utente e manuale tecnico, javadoc

Setup

Prima di aprire la sottocartella bin da terminale e avviare l'applicazione con il comando `java -jar CineMax.jar`, è necessario verificare che il proprio ambiente rispetti le seguenti condizioni:

- La cartella CineMax-main deve essere posta in una directory normale del sistema, evitando percorsi protetti (ad es. Program Files)
- La versione Java installata deve essere una JDK di versione pari o superiore alla 21, la release stabile più datata con cui l'applicazione è stata sviluppata; il corretto funzionamento nelle precedenti versioni non è stato testato

Struttura e Funzionalità

1. Architettura a Layer

Il sistema CineMax adotta un'architettura software stratificata a cinque layer: **cinemax**, **textuserinterface**, **service**, **repository** e **model**. È presente inoltre un layer trasversale **utility**, accessibile dai tre layer superiori.

Layer	Responsabilità
cinemax	Entry point dell'applicazione, configurazione e avvio.
textuserinterface	Gestione dell'interazione testuale con l'utente.
service	Logica di business.
repository	Persistenza e accesso ai dati su file.
model	Definizione delle entità del dominio.

2. Layer Model

Il layer model contiene le classi che rappresentano le entità del dominio applicativo e le strutture dati utilizzate per la presentazione delle informazioni. Le classi si suddividono in due categorie funzionalmente distinte: le entità persistibili e le entità di presentazione.

2.1 Interfacce di Contratto

Due interfacce generiche definiscono il contratto che le entità persistibili devono rispettare, garantendo la coerenza con il layer di persistenza.

Identifiable<E>

L'interfaccia `Identifiable<E>` definisce il contratto per le entità che possiedono un identificativo univoco. Espone due metodi: `getId()`, che restituisce l'identificativo tipizzato dell'oggetto, e `getFields()`, che restituisce un array di `String` contenente tutti i campi dell'entità nell'ordine di serializzazione. Quest'ultimo metodo è fondamentale per la persistenza: è la controparte di deserializzazione offerta dai costruttori che accettano `String[]`.

ItemInitializer<E>

L'interfaccia `ItemInitializer<E>` definisce il metodo factory `getNewItem(String[] array)`, che istanzia e restituisce un nuovo oggetto del tipo generico `E` a partire da un array di stringhe. Questo pattern consente a `GenericRepository` di deserializzare le righe del file senza dipendere da un tipo concreto, operando esclusivamente sull'interfaccia.

2.2 Entità Persistibili

Le entità persistibili implementano entrambe le interfacce `ItemInitializer` e `Identifiable`. Ogni classe espone due costruttori complementari: uno con parametri tipizzati per la creazione programmatica, e uno che accetta un `String[]` per la

deserializzazione da file. Il metodo `getFields()` funge da serializzatore, producendo l'array corrispondente da rendere persistente.

Classe	ID (tipo)	Campi	Ruolo nel Dominio
Movie	Long (movieId)	title, director, year, genre, runningTime, minAge	Rappresenta un film nel catalogo.
Show	Long (showId)	movieId (FK), showDate, ticketCost	Rappresenta una proiezione di un film.
Reservation	Long (reservationId)	username (FK), showId (FK), ticketsNumber	Rappresenta una prenotazione di un Utente.
User	String (username)	name, surname, password, birthDate, residence, role	Rappresenta un utente del sistema.

Movie

Modella un film del catalogo cinematografico con i relativi dati. Utilizza tipi numerici compatti (`short` per anno e durata, `byte` per l'età minima) per limitare lo spazio occupato in memoria. L'id è di tipo `Long`.

Show

Modella una proiezione, collegando un film (tramite `movieId`) a una data/ora (`LocalDateTime`) e a un costo del biglietto (`float`). Rappresenta l'evento concreto prenotabile dall'utente. La data viene serializzata tramite `LocalDateTime.toString()` e deserializzata con `LocalDateTime.parse()`.

Reservation

Modella una prenotazione, associando lo `username` di un utente allo `showId` di una proiezione con il relativo numero di biglietti (`short`). Funge da tabella di associazione tra `User` e `Show`.

User

Modella un utente del sistema con i suoi dati anagrafici e il ruolo. Il campo `role` è di tipo `Role` (enum). La deserializzazione del ruolo avviene tramite uno `switch` sul valore stringa. L'id è lo `username (String)`, che è l'unico caso in cui l'identificativo non è numerico.

`role` è un'enumerazione che definisce i quattro ruoli del sistema: `CLIENT`, `PROJECTIONIST`, `BOXOFFICECLERK` e `ADMIN`. A runtime il ruolo determina il tipo di menu (`textuserinterface`), con le relative operazioni (`service`), visualizzato dall'utente.

2.3 Entità di Presentazione

Le entità di presentazione sono classi a logica puramente di visualizzazione. Non implementano `Identifiable` né `ItemInitializer` e non vengono mai rese persistenti. Vengono istanziate dal layer service aggregando dati da più entità persistibili, e il loro unico

scopo è produrre una rappresentazione testuale strutturata tramite il metodo `toString()`. I campi sono tutti di tipo `String`, incluse le date già formattate con pattern `dd/MM/yyyy HH:mm`.

Classe	Sorgenti Aggregate	Informazioni Esposte
ShowDetails	Show + Movie	id proiezione, titolo, data/ora, costo biglietto
FullShowDetails	Show + Movie + freeSeats	Tutti i dati di ShowDetails + genere, regista, anno, durata, età minima, posti disponibili
ReservationDetails	Reservation + User + Show + Movie	id prenotazione, nome, cognome, data/ora, titolo film
FullReservationDetails	Reservation + User + Show + Movie	Tutti i dati di ReservationDetails + numero biglietti, costo biglietto, costo totale

La distinzione tra variante Details e variante Full permette di presentare solo le informazioni strettamente necessarie: la versione ridotta viene usata nei contesti dove il riepilogo è sufficiente (es. lista prenotazioni), mentre la versione completa viene usata nei contesti dove è necessario l'insieme esaustivo dei dati (es. dettaglio della singola prenotazione o proiezione).

3. Layer Repository

Il layer repository è responsabile della persistenza delle entità su file di testo. L'architettura presenta tre componenti: un'interfaccia generica che definisce il contratto, repository specifici per tipo di entità che implementano l'interfaccia e un repository generico sottostante che fornisce le implementazioni concrete parametrizzate.

3.1 Interfaccia FileRepository<F, ID>

L'interfaccia `FileRepository<F, ID>` definisce il contratto pubblico per l'accesso ai dati, parametrizzando sia il tipo dell'entità (`F`) che il tipo del suo identificativo (`ID`). Espone le seguenti operazioni:

Metodo	Descrizione
findById(ID id)	Ricerca e restituisce l'entità con l'id specificato, null se assente.
insert(F entity)	Inserisce una nuova entità, restituisce false se l'id è già presente.
delete(ID id)	Elimina l'entità con l'id specificato, restituisce false se assente.
update(F entity)	Rende persistenti le modifiche dell'entità in argomento, restituisce false se assente.
getMaxId()	Restituisce l'id massimo presente nel repository.
startSequentialReading()	Inizializza una sessione di lettura sequenziale aprendo il FileChannel.
endSequentialReading()	Chiude il FileChannel della sessione corrente di lettura sequenziale.
getNextItems()	Restituisce il prossimo blocco di entità nella lettura sequenziale, null se a fine file.

La tripletta `startSequentialReading()` / `getNextItems()` / `endSequentialReading()` crea un'interfaccia di lettura in stile iteratore, che consente di scorrere l'intero contenuto del file a blocchi senza caricarlo interamente in memoria.

3.2 GenericRepository<E, F>

La classe `GenericRepository<E extends Comparable<E>, F extends ItemInitializer<F> & Identifiable<E>>` è il nucleo del layer di persistenza. Implementa tutte le operazioni di I/O su file utilizzando `FileChannel` e `ByteBuffer` della NIO API di Java, operando su un file di testo dove ogni riga rappresenta un'entità e i campi sono separati dal carattere tilde (~).

3.2.1 Struttura del File e Parametri

Ogni entità è serializzata su una singola riga terminata da carattere linefeed (\n). I campi sono separati dal carattere tilde. La lettura avviene per blocchi (pagine) di dimensione fissa. Nell'implementazione attuale quest'ultimo parametro ha valore pari a 16 KB, per lavorare con un numero di byte abbastanza grande da ridurre il numero di accessi al disco, ma ridotto a sufficienza per entrare in cache.

Costante	Valore	Significato
PAGE_DIMENSION	16.384 byte	Dimensione del buffer di lettura per ogni operazione I/O.
DIVIDER	'~'	Separatore tra i campi di un'entità serializzata.
TERMINATOR	'\n'	Terminatore di riga (separatore tra entità).

3.2.2 Analisi della Complessità

Sia n il numero di entità presenti nel file e s la dimensione media in byte di una riga serializzata. Il numero di operazioni I/O è proporzionale a P , la parte intera superiore di $(n \cdot s) / \text{PAGE_DIMENSION}$.

Metodo	Complessità	Motivazione
findByld(id)	$O(n)$	Scansione sequenziale del file: legge blocchi da <code>PAGE_DIMENSION</code> byte fino a trovare l'id o raggiungere EOF. Nel caso peggiore percorre l'intero file.
save(item)	$O(n)$ caso peggiore, $O(1)$ caso ottimale	Se l'id da inserire è strettamente maggiore del massimo corrente (caso frequente con id auto-incrementali), l'append è $O(1)$. In caso contrario viene invocato <code>findByld</code> per verificare i duplicati: $O(n)$.
update(item)	$O(n)$	Richiede <code>findByld</code> per verificare l'esistenza dell'elemento ($O(n)$), poi una riscrittura completa del file su file temporaneo tramite <code>modify()</code> .
delete(id)	$O(n)$	Analogo a <code>update</code> : verifica esistenza dell'elemento con <code>findByld</code> ($O(n)$), poi riscrive l'intero file escludendo la riga eliminata.

getMaxId()	O(n) alla prima chiamata o dopo delete della tupla con id massimo, O(1) altrimenti	Alla prima invocazione scansiona l'intero file per trovare il massimo. Il risultato viene memorizzato in cache (campo maxId): le chiamate successive restituiscono il valore direttamente dall'apposito campo. Una nuova ricerca lineare è necessaria se la cache viene invalidata, cioè dopo l'eliminazione della tupla con id massimo.
getNextItems()	O(P) per chiamata	Legge un blocco da PAGE_DIMENSION byte per invocazione, gestendo tuple incomplete a cavallo di blocchi tramite il buffer incompleteTuple. Il caso peggiore è la lettura di un file costituito da una sola tupla estesa su più blocchi, dove sono necessarie esattamente P letture perché la chiamata restituisca l'elemento.

3.2.3 Meccanismo di Lettura a Blocchi

Immaginando di lavorare con blocchi contenenti ciascuno c tuple, la lettura sequenziale opera in complessità spaziale $\Theta(c)$, a prescindere dalla dimensione del file. In uno scenario non ideale e ragionando sul comportamento peggiore, un file contenente una sola tupla estesa porterebbe il metodo `getNextItems()` a concatenare più blocchi, fino a dover gestire in memoria l'intero file. Questo, però, è poco plausibile.

Il metodo `getNextItems()` legge un blocco di PAGE_DIMENSION byte per volta. Il metodo privato `getTuples(ByteBuffer)` analizza il buffer alla ricerca di caratteri terminatori (`\n`), estraendo le tuple complete. Un'eventuale tupla parziale a fine blocco (stringa priva di terminatore) viene salvata nel campo `incompleteTuple` e preposta al blocco successivo. Questo meccanismo gestisce correttamente le entità a cavallo di blocchi consecutivi.

3.2.4 Meccanismo di Modifica e Cancellazione

Le operazioni `update()` e `delete()` non modificano il file in-place, ma adottano il pattern **copy-on-write**: il contenuto viene riscritto su un file temporaneo (`temp_<filename>`), escludendo o sostituendo la riga dell'entità target, dopodiché il file temporaneo sostituisce quello originale tramite `Files.move()` con opzione `REPLACE_EXISTING`.

Il metodo privato `modify(E id, String content, FileChannel iChannel, FileChannel oChannel)` gestisce la logica di copia: per ogni blocco letto, ricostruisce le tuple e scrive sul canale di output tutte quelle il cui id è diverso da quello target; se il target viene trovato e se `content` è non nullo, scrive la versione aggiornata in sostituzione, mentre se è nullo (caso `delete`) omette semplicemente il target.

Il metodo `writeData()` ottimizza la scrittura garantendo che i blocchi in output siano anch'essi di dimensione PAGE_DIMENSION quando possibile, ritardando la scrittura di un eventuale blocco troppo piccolo (`tooShortBlock`), che invece viene ampliato con dati dal blocco di lettura successivo.

3.2.5 Cache del MaxId

Il campo privato `maxId` funge da cache dell'identificativo massimo e viene inizializzato a `null`. Il metodo `getMaxId()` popola la cache alla prima invocazione con una scansione completa ($O(n)$); le successive restituiscono il valore in $O(1)$. La cache viene invalidata

(reimpostata a `null`) solo in caso di `delete()` dell'elemento con id massimo. L'operazione `save()` aggiorna la cache direttamente se il nuovo id è maggiore del massimo corrente.

3.3 Repository Specifici

I repository concreti `MovieRepository`, `ShowRepository`, `ReservationRepository` e `UserRepository` implementano `FileRepository`, delegando ogni operazione a un'istanza interna di tipo `GenericRepository` opportunamente parametrizzato.

Repository	Tipo Entità	Tipo ID	File
MovieRepository	Movie	Long	movie_repository.txt
ShowRepository	Show	Long	show_repository.txt
ReservationRepository	Reservation	Long	reservation_repository.txt
UserRepository	User	String	user_repository.txt

Si noti che `UserRepository` è l'unico repository con identificativo di tipo `String`: lo `username` è una stringa che implementa `Comparable<String>`, soddisfacendo il vincolo del parametro `E` `extends Comparable<E>` di `GenericRepository`. Il confronto lessicografico tra stringhe sostituisce quello numerico negli algoritmi di ricerca dell'id massimo.

Attenzione - Vale la pena osservare che l'implementazione attuale non prevede invocazioni di `getMaxId()` su `UserRepository`. Nondimeno, per uniformità dei metodi esposti dalle repository specifiche, `getMaxId()` è stato incluso nell'interfaccia `FileRepository` e inserito in ogni classe che la implementa.

3.4 Gestione delle Eccezioni

La classe `FileException` estende `RuntimeException` e viene lanciata da `GenericRepository` in tutti i casi in cui un'operazione I/O su file fallisce (creazione, apertura, lettura, scrittura, spostamento). Essendo unchecked, non obbliga il chiamante a gestirla esplicitamente, ma la sua propagazione verso i layer superiori (`service`, `textuserinterface`) consente la gestione centralizzata degli errori di persistenza.

4. Layer Utility

Il layer utility è trasversale: le sue classi sono accessibili dai layer `service`, `textuserinterface` e `cinemax`, ma non dai layer `repository` e `model`. Contiene funzionalità di supporto che non appartengono a nessun singolo layer, ma che sono condivise da più componenti del sistema.

4.1 OutputPrinter

La classe `OutputPrinter` centralizza la formattazione di tutto l'output testuale del sistema. Viene costruita con tre parametri: una stringa di margine standard (`margin`), una stringa di

margin per le voci di riepilogo e le opzioni (`marked`) e una lunghezza massima di riga (`rowLength`). Espone tre metodi pubblici:

Metodo	Comportamento
<code>print(String)</code>	Stampa il testo preceduto dal margine standard, senza linefeed finale.
<code>println(String)</code>	Come <code>print</code> , con linefeed finale.
<code>printlnMarked(String)</code>	Stampa con il margine marcato per la prima riga non vuota, margine standard per le restanti righe; aggiunge linefeed finale.
<code>printlnMarkedByChunk(List<String>)</code>	Tratta ogni singola stringa come farebbe la <code>printlnMarked</code> , ma stampa in blocco unico alla fine dell'elaborazione complessiva.

Il metodo privato `printWithMargin()` gestisce la logica di word-wrap: se una riga supera `rowLength` caratteri, la tronca all'ultimo spazio utile e gestisce i caratteri rimanenti su una nuova riga (facendoli precedere dal margine standard). Spazi multipli consecutivi vengono collassati a uno singolo. Questo garantisce un output coerente e leggibile indipendentemente dalla lunghezza delle stringhe prodotte dagli strati inferiori.

4.2 PasswordHandler

La classe `PasswordHandler` gestisce la sicurezza delle credenziali tramite l'algoritmo PBKDF2WithHmacSHA256 con 600.000 iterazioni, chiave da 256 bit e salt casuale da 16 byte generato con `SecureRandom`. La password verrà salvata dal layer repository come stringa `saltBase64:hashBase64`. La verifica utilizza un confronto a tempo costante (`slowEquals`) per prevenire attacchi di tipo timing. I metodi statici `hashPassword()` e `verifyPassword()` dichiarano le eccezioni checked `NoSuchAlgorithmException` e `InvalidKeySpecException`, che il layer service cattura e ritraduce in `SafetyException`.

4.3 SafetyException

La classe `SafetyException` estende `RuntimeException` e rappresenta un errore fatale e non recuperabile legato alla sicurezza del sistema (es. algoritmo di hashing non disponibile). A differenza di `PromptException` (che segnala errori di input gestibili), una `SafetyException` viene propagata fino al `main`, che la intercetta e termina l'esecuzione con un messaggio di errore.

5. Layer Service

Il layer service implementa tutta la logica di business dell'applicazione. Le sue classi dipendono dai repository e da `OutputPrinter` tramite iniezione nel costruttore, senza istanziarle direttamente. Ogni metodo pubblico di service corrisponde a un'operazione di

interrogazione o manipolazione delle basi di dati, combinando lettura e scrittura sui repository sottostanti.

Il layer service definisce anche l'eccezione `PromptException`: una `RuntimeException` unchecked che viene lanciata quando una pre-condizione di business non è soddisfatta (es. proiezione inesistente, posti insufficienti, prenotazione altrui). Non rappresenta un errore di sistema, ma un esito negativo prevedibile di un'operazione richiesta dall'utente: il layer `textuserinterface` la cattura e ne visualizza il messaggio senza interrompere il ciclo dell'applicazione.

5.1 Authentication

La classe `Authentication` gestisce il ciclo di vita delle credenziali e dei ruoli utente.

Dipende esclusivamente da `UserRepository`. I metodi principali sono:

Metodo	Descrizione
<code>isFirstAccess()</code>	Verifica se il repository utenti è vuoto, per forzare la creazione del primo account admin.
<code>signIn(username, password)</code>	Cerca l'utente per username; se trovato, verifica la password con <code>PasswordHandler</code> ; restituisce l'oggetto <code>User</code> o lancia <code>PromptException</code> .
<code>signUp(String username, String name, String surname, String password, LocalDate birthDate, String residence)</code>	Verifica l'unicità dello username, fa l'hash della password e inserisce un nuovo utente con ruolo CLIENT.
<code>adminSignUp(String username, String name, String surname, String password, LocalDate birthDate, String residence)</code>	Analogo a <code>signUp</code> ma assegna il ruolo ADMIN; usato solo al primo accesso.
<code>makeProjectionist(username), makeBoxOfficeClerk(username)</code>	Recupera l'utente, verifica che non sia ADMIN, aggiorna il ruolo e rende persistente la modifica tramite <code>update</code> .

5.2 MovieService

La classe `MovieService` gestisce le operazioni sul catalogo dei film. Dipende da `MovieRepository` e `OutputPrinter`; come nei service successivi, la dipendenza da `OutputPrinter` è necessaria per la stampa in streaming, discussa nella sezione 5.5.

Metodo	Descrizione
<code>searchAndPrintMovies(title, director, year)</code>	Scansione sequenziale del catalogo con filtri opzionali su titolo (ricerca parziale case-insensitive), regista (match case-insensitive) e anno. Stampa ogni risultato in streaming.
<code>addMovie(title, director, year, genre, runningTime, minAge)</code>	Verifica se esiste già un film identico per tutti i campi (deduplicazione); in caso positivo restituisce l'id esistente, altrimenti inserisce il nuovo film e restituisce l'id assegnato.

La logica di `addMovie()` implementa una deduplicazione implicita: il proiezionista può digitare i dati di un film già nel catalogo e il sistema lo riconosce, restituendo l'id esistente senza duplicarlo.

Attenzione - Il metodo, in caso di aggiunta tramite campi, può essere molto costoso perché nello scenario peggiore, cioè quando il film da aggiungere non compare nella base di dati, scorre l'intero file e scansiona attributo per attributo ogni tupla.

5.3 ShowService

La classe `ShowService` gestisce le proiezioni. Dipende da `ShowRepository`, `ReservationRepository`, `MovieRepository` e `OutputPrinter`. La funzione di ricerca che espone è comune al cliente e al proiezionista.

Metodo	Descrizione
<code>searchAndPrintShows(...)</code>	Scansione sequenziale con filtri opzionali su titolo (ricerca parziale case-insensitive), genere (match case-insensitive), intervallo di date e range di prezzo. Stampa <code>ShowDetails</code> in streaming.
<code>visualizeShow(showId)</code>	Recupera la proiezione, calcola i posti liberi (200 meno i biglietti prenotati) e restituisce un oggetto <code>FullShowDetails</code> ; restituisce null se la proiezione è assente.
<code>addShow(movieId, showDate, ticketCost)</code>	Verifica che il film esista e che la nuova proiezione non si sovrapponga temporalmente ad alcuna proiezione esistente, poi procede all'aggiunta.
<code>editShow(showId, newDate, newCost)</code>	Modifica data e/o costo di una proiezione, solo se non esistono prenotazioni associate.
<code>deleteShow(showId)</code>	Elimina una proiezione solo se priva di prenotazioni associate.

5.4 ReservationService

La classe `ReservationService` è la più articolata del layer: coordina tutti e quattro i repository e implementa la logica di business relativa alle prenotazioni, incluse le operazioni sia per il cliente sia per il bigliettaio.

Metodo	Descrizione
<code>addReservation(username, showId, ticketsNumber)</code>	Verifica esistenza e data e ora della proiezione (non deve essere già avvenuta), controlla duplicati per lo stesso utente, verifica disponibilità posti (max 200), inserisce la prenotazione.
<code>editReservation(username, reservationId, newShowId, ticketsNumber)</code>	Verifica proprietà della prenotazione, data e ora di entrambe le proiezioni (vecchia e nuova non devono essere già avvenute), disponibilità posti sulla nuova. Gestisce separatamente il caso di cambio proiezione vs. stessa proiezione.
<code>deleteReservation(username, reservationId)</code>	Verifica proprietà e permette l'eliminazione solo di prenotazioni relative a proiezioni già avvenute.

<code>printMyReservations(username)</code>	Stampa in streaming le prenotazioni dell'utente corrente con <code>FullReservationDetails</code> .
<code>printTodayReservations()</code>	Scansione di tutte le prenotazioni, filtro su anno e giorno correnti, stampa in streaming di <code>ReservationDetails</code> .
<code>searchAndPrintReservations(...)</code>	Ricerca per filtri (id, nome, cognome, titolo parziale, intervallo date) con stampa di <code>ReservationDetails</code> in streaming.
<code>visualizeReservation(reservationId)</code>	Restituisce un oggetto <code>FullReservationDetails</code> per la prenotazione richiesta, null se assente.
<code>countTicketsByShow(rRepo, showId)</code>	Metodo statico di utilità: somma i biglietti di tutte le prenotazioni per una data proiezione. Utilizzato da <code>ShowService.visualizeShow(showId)</code> .

5.5 Scelta Architettureale: Stampa in Streaming nel Layer Service

Nota di progetto: I metodi `searchAndPrintMovies`, `searchAndPrintShows`, `printMyReservations`, `printTodayReservations` e `searchAndPrintReservations` violano parzialmente la separazione dei ruoli tra layer: delegare la stampa al layer service introduce una dipendenza da `OutputPrinter` in un layer che dovrebbe occuparsi esclusivamente di logica di business. Questa scelta è tuttavia consapevole e motivata da un preciso requisito di efficienza.

Un'alternativa che rispetti la separazione dei layer prevede in teoria che i metodi di ricerca restituiscano al layer `textuserinterface` una `List<F>` completa dei risultati. Il layer TUI deve poi occuparsi della stampa. Questa soluzione, tuttavia, richiede di accumulare in memoria l'intera lista dei risultati prima di poterne visualizzare anche solo il primo elemento, con complessità spaziale $\Theta(n)$ rispetto al numero totale di entità nel repository.

L'approccio adottato integra invece la stampa direttamente all'interno del ciclo di lettura a blocchi reso possibile da `getNextItems()` di `GenericRepository`: ogni blocco di entità restituito da tale metodo viene filtrato e stampato immediatamente, senza mai accumulare l'intera collezione. La complessità spaziale si riduce così a $\Theta(c)$, dove c è il numero medio di tuple presenti in un singolo blocco restituito da `getNextItems()` (determinato dal rapporto tra `PAGE_DIMENSION` e la dimensione media di una riga serializzata). Indipendentemente dalla dimensione totale del repository, la memoria occupata a runtime per la lista dei risultati è virtualmente costante.

Questo pattern produce inoltre un effetto di output in streaming: i risultati vengono visualizzati progressivamente durante la scansione, senza che l'utente debba attendere il completamento dell'intera lettura del file. Il costo architetturale (accoppiamento di service con `OutputPrinter`) è quindi una scelta consapevole, giustificata dal guadagno di efficienza spaziale e dalla miglior esperienza utente per dataset di grandi dimensioni e letti a scorrimento.

6. Layer TextUserInterface

Il layer `textuserinterface` è responsabile di tutta l'interazione con l'utente: raccoglie gli input, presenta i menu e delega ogni elaborazione al layer `service`. Non contiene logica di business né accede direttamente ai repository. Riceve le sue dipendenze — i quattro `service` e `OutputPrinter` — per iniezione nel costruttore di `TextUserInterface`.

6.1 MenuHelper

La classe `MenuHelper` centralizza lettura e validazione dell'input da console, incapsulando un'istanza statica di `Scanner`. Ogni metodo di lettura implementa un ciclo di retry che continua a chiedere l'input fino a quando questo non soddisfa i requisiti, stampando un messaggio di errore ad ogni tentativo fallito. Il parametro `mandatory` (presente in tutti i metodi tranne `readInt`) determina se il campo è obbligatorio: se `false`, un input vuoto viene accettato e tradotto in `null`, consentendo filtri opzionali nelle ricerche demandate a `service`.

Metodo	Tipo restituito / Validazione
<code>readInt(msg, min, max)</code>	<code>int</code> — valore nel range <code>[min, max]</code> ; nessun <code>null</code>
<code>readPositiveByte(msg, mandatory)</code>	<code>Byte</code> — intero > 0 ; <code>null</code> se non obbligatorio e input vuoto
<code>readPositiveShort(msg, mandatory)</code>	<code>Short</code> — intero > 0 ; <code>null</code> se non obbligatorio e input vuoto
<code>readLong(msg, mandatory)</code>	<code>Long</code> — intero ≥ 0 ; <code>null</code> se non obbligatorio e input vuoto
<code>readFloat(msg, mandatory)</code>	<code>Float</code> — numero ≥ 0 ; <code>null</code> se non obbligatorio e input vuoto
<code>readString(msg, mandatory)</code>	<code>String</code> — rifiuta caratteri <code>~</code> e <code> </code> (riservati ai layer di persistenza e dei modelli); <code>null</code> se non obbligatorio e input vuoto
<code>readDate(msg, mandatory)</code>	<code>LocalDate</code> — formato <code>dd/MM/yyyy</code> ; <code>null</code> se non obbligatorio e input vuoto
<code>readDateAndTime(msg, mandatory)</code>	<code>LocalDateTime</code> — formato <code>dd/MM/yyyy HH:mm</code> ; <code>null</code> se non obbligatorio e input vuoto

La validazione sui caratteri `~` e `|` in `readString()` è una misura di protezione dell'integrità nel layer di persistenza e dell'univocità delle rappresentazioni nel layer `model`: il carattere `~` è il separatore di campo usato da `GenericRepository`, mentre `|` è usato dal metodo `toString()` delle entità `model`. Permetterne l'inserimento causerebbe una corruzione del file al momento della serializzazione o la perdita di chiarezza nell'output da console.

6.2 TextUserInterface

La classe `TextUserInterface` gestisce il ciclo principale dell'applicazione e i menu per ciascun ruolo. Mantiene un campo `currentUser` (di tipo `User`, inizialmente `null`) che rappresenta l'utente autenticato. Al `logout`, il campo viene invalidato, riportando l'applicazione al menu ospite.

Ciclo Principale e Dispatch per Ruolo

Il metodo `start()` esegue un loop infinito: ad ogni iterazione, se `currentUser` è `null` viene mostrato il menu ospite; altrimenti viene eseguito uno switch sul ruolo dell'utente corrente per selezionare il menu appropriato. Ogni menu presenta le opzioni disponibili, legge la scelta con `readInt()` e invoca il metodo privato corrispondente.

Menu (ruolo)	Operazioni disponibili
Ospite (null)	Cerca proiezioni, visualizza dettaglio proiezione, registrazione, login, esci.
CLIENT	Cerca proiezioni, visualizza dettaglio proiezione, aggiungi/visualizza/modifica/cancella prenotazioni, logout.
PROJECTIONIST	Filtra film, filtra proiezioni, aggiungi/modifica/elimina proiezione, logout.
BOXOFFICECLERK	Cerca prenotazioni, visualizza prenotazioni odierne, visualizza dettaglio prenotazione, logout.
ADMIN	Promuovi a proiezionista, promuovi a bigliettaio, logout.

Gestione delle Eccezioni

Tutti i metodi privati che invocano operazioni di service il cui input deve essere validato wrappano la chiamata in un blocco `try/catch (PromptException)`. In caso di eccezione, il messaggio viene stampato e il controllo ritorna al menu corrente, senza interrompere il ciclo dell'applicazione. Le eccezioni `FileNotFoundException` e `SafetyException` non vengono gestite a questo livello, ma si propagano fino al `main`.

getAdminPanel()

Il metodo `getAdminPanel()` viene invocato come prima istruzione in `start()`, prima che questo avvii il loop descritto precedentemente. Se `isFirstAccess()` restituisce `true` (repository utenti vuoto), forza la creazione interattiva del primo account amministratore tramite `adminSignUp()`. Questo meccanismo garantisce che il sistema non sia mai avviabile in assenza di un account admin, senza richiedere configurazione manuale dei file.

7. Layer Cinemax (Main)

Il layer cinemax è composto dalla sola classe `CineMax`, che contiene il metodo `main` e le costanti globali per la formattazione dell'output. Il suo unico ruolo è orchestrare l'inizializzazione di tutti i componenti e avviare il ciclo di interazione.

7.1 Sequenza di Inizializzazione

Il metodo `main` segue una sequenza di inizializzazione bottom-up, rispettando le dipendenze tra i layer:

- Viene creata un'istanza di `OutputPrinter` con le costanti di formattazione definite nella classe (margine, marcatore, lunghezza riga).
- Vengono istanziati i quattro repository concreti. In questa fase, `GenericRepository` verifica l'esistenza dei file di dati e li crea se assenti.

- Vengono istanziati i service, ognuno con le relative dipendenze sui repository e su `OutputPrinter`.
- Viene istanziata `TextUserInterface` con i quattro service e `OutputPrinter`.
- Viene invocato `tui.start()`, che avvia il loop infinito di interazione.

L'intero blocco di inizializzazione è wrappato in un `try/catch (FileNotFoundException | SafetyException)`: qualunque errore fatale (impossibilità di creare o accedere ai file di dati, o errore critico nel sistema crittografico) viene catturato qui, stampato tramite `OutputPrinter` e causa la terminazione dell'applicazione. Questa è l'unica sede dove tali eccezioni vengono gestite.

7.2 Costanti Globali

Costante	Valore	Utilizzo
MARGIN	" " (8 spazi)	Margine standard di ogni riga di output
MARKER	" * " (6 spazi + * + 1 spazio)	Margine per la prima riga (non vuota) delle opzioni o dei riepiloghi (<code>printlnMarked</code>)
ROW_LENGTH	76 caratteri	Lunghezza massima del contenuto testuale per riga, prima dell'anteposizione del margine

Centralizzare le costanti nel `main` garantisce un unico punto di controllo per l'aspetto visivo dell'intera applicazione.

Limiti della Soluzione Sviluppata

Parsing Fragile

I file per la persistenza delle basi di dati vengono scansionati a scorrimento, estraendo le tuple grazie al riconoscimento del carattere terminatore ('\n') e i campi grazie al carattere divisore (~). `GenericRepository` non esegue alcuna verifica aggiuntiva, quindi basta l'invalidazione di uno solo di questi caratteri per rendere illeggibile una porzione o addirittura l'interezza del file. Per questa ragione, si sconsiglia la modifica manuale dei file persistenti, soprattutto se si sta lavorando senza backup e con dataset non di prova.

Lentezza degli Scorrimenti

Per quanto le operazioni di I/O avvengano a blocchi e non a singole tuple, la logica di recupero dei dati è sempre a scorrimento. Funzioni di ricerca con filtro su più basi di dati alla volta, come `searchAndPrintShows` (scansione di proiezioni e film), `printMyReservations` (scansione di prenotazioni, proiezioni, film e utenti), `printTodayReservations` (come prima) e `searchAndPrintReservations` (come prima) nel Layer service sono operazioni con complessità temporale $O(n_1 * n_2 * \dots * n_m)$, dove ogni n_i rappresenta il numero complessivo di elementi nella base di dati i -esima. Per basi di dati estese, le operazioni calano drasticamente in velocità.

In assenza di un appoggio ad un DBMS remoto, una soluzione viabile è sostituire la logica di scorrimento della `findById` in `GenericRepository` con una ricerca binaria o con il ricorso ad un indice hash caricato in RAM.

Il volume ridotto dei dati previsto nei file, dato il ristretto dominio applicativo rappresentato da un cinema monosala da 200 posti, ha reso preferibile la semplicità dell'attuale implementazione rispetto a versioni più efficienti.